47. Calling Model APIs: HTTP and SSE Streaming

47. Calling Model APIs: HTTP and SSE Streaming

Phase: Phase 7 — AI engineering in Go
Estimated time: 2 days
Slug: `47-model-apis`

Learning objectives

- Call LLM HTTP APIs with timeouts
- Parse streaming SSE responses
- Handle rate limits and retries
- Bound concurrency to model backends

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Model APIs stream tokens via HTTP/SSE. Go clients must parse streams and cancel cleanly when users abort.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- SSE: lines of `data:` events separated by blank lines.
- Decode incrementally from a Reader.
- Honor context cancel mid-stream.
- Retries differ for streaming vs unary calls.
- Auth headers and timeouts still apply.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-calling && cd /tmp/gw-calling
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "bufio"
    "fmt"
    "strings"
)

func ParseDataLines(s string) []string {
    var out []string
    sc := bufio.NewScanner(strings.NewReader(s))
    for sc.Scan() {
        line := sc.Text()
        if strings.HasPrefix(line, "data:") {
            out = append(out, strings.TrimSpace(strings.TrimPrefix(line,
"data:")))
        }
    }
    return out
}

func main() {
    raw := "data: hi\n\ndata: there\n\n"
    fmt.Println(ParseDataLines(raw))
}
```

Run it:

```
go run .
```

Expected output:

```
[hi there]
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Parse a multi-event SSE string.
2. Stop parsing when ctx is done (sketch).
3. Handle `[DONE]` sentinel used by some APIs.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Request shape

Most model APIs are HTTPS JSON. Stream tokens via Server-Sent Events (SSE) or chunked HTTP.

```
req, _ := http.NewRequestWithContext(ctx, http.MethodPost, url, body)
req.Header.Set("Authorization", "Bearer "+key)
req.Header.Set("Accept", "text/event-stream")
```

SSE parsing

Read line-oriented `data: ...` frames until `[DONE]`. Respect ctx cancellation mid-stream.

Production concerns

- Token/cost accounting
- Prompt injection awareness at boundaries
- Never log raw secrets or full prompts if sensitive

Failure modes to watch while coding

- No timeout on stream reads
- Buffering entire stream before forwarding
- Unbounded parallel calls to the model

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- httpx/stream vs bufio.Scanner.
- Same SSE framing.
- Generators yield chunks; Go channels can too.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Always test with partial reads — real networks fragment events.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Calling Model APIs: HTTP and SSE Streaming” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- No timeout on stream reads
- Buffering entire stream before forwarding
- Unbounded parallel calls to the model

Exercises

Work in `exercises/47-model-apis/`. Read that folder's `README.md` before you code.

What you will build

Parse SSE data lines as the exercise tests specify.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/47-model-apis/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What is SSE?
2. Why cancel mid-stream?
3. Which headers matter?

Further reading

- https://platform.openai.com/docs/api-reference